Lecture 22: Newton's Method and Projected Gradient Method

Zizhuo Wang

School of Data Science (SDS)
The Chinese University of Hong Kong, Shenzhen

Apr 26, 2023

Announcement

- Homework 8 due on Apr 28

Recap

We have introduced several methods for unconstrained optimization problems

For one-dimensional problems

- Bisection method and golden section method

For high-dimensional problems, we first studied the gradient descent method

- Choose search direction as the negative of the gradient
- Stepsize: Exact line search and backtracking line search (the latter one is preferred in practice)
- Stopping criterion: Based on the norm of the gradient
- Convergence: Linear convergence
- Other properties: Globally convergence, perpendicular search direction when using exact line search

Armijo Condition

In backtracking line search, the stopping condition

$$f(\mathbf{x}^k + t\mathbf{d}^k) \leq f(\mathbf{x}^k) + \alpha t \nabla f(\mathbf{x}^k)^T \mathbf{d}^k$$

is also called the Armijo Condition.

Newton's Method

In the one-dimensional case

$$x^{k+1} = x^k - \frac{g(x^k)}{g'(x^k)}$$

where $g(x) = f'(x)$.

Two ways to view Newton's method

- Approximate the derivative function locally by a linear function
- Approximate the original function locally by a quadratic function

Convergence of Newton's Method (for 1-Dimension Case)

Theorem

If $g(x)$ is twice continuously differentiable and x^* is a root of $g(x)$ at which $g'(x^*) \neq 0$, then provided that $|x^0 - x^*|$ is sufficiently small, the sequence generated by the Newton iterations:

$$x^{k+1} = x^k - \frac{g(x^k)}{g'(x^k)}$$

will satisfy

$$|x^{k+1} - x^*| \leq C|x^k - x^*|^2$$

with $C = \sup_x \frac{1}{2}|\frac{g''(x)}{g'(x)}|$.

- We call this convergence speed *quadratic convergence*—much faster then linear convergence

Newton's Method in High Dimensional Case

By using the second-order Taylor expansion, we obtained the Newton's step in the high-dimensional case as:

$$\mathbf{x} = \mathbf{x}^k - (\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$$

- We showed that this step is a descent direction when f is convex.

As we said earlier, Newton's method may not converge unless the starting point is close.

- One way to help convergence is to use a step size parameter α_k in

$$\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$$

  where $\mathbf{d}^k = -(\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$ is the Newton's step.
- One can use backtracking line search to determine α_k

Complete Procedure of Newton's Method

Start from a point $\mathbf{x}^0$, set tolerance $\epsilon > 0$ and $k = 0$.

1. Compute the Newton's step:

$$\mathbf{d}^k = -(\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$$

   If $||\nabla f(\mathbf{x}^k)|| < \epsilon$, then output $\mathbf{x}^k$, otherwise continue;
2. Choose step size α_k by backtracking line search
3. $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$, $k = k + 1$, go back to step 1

Convergence Results for Newton's Method

Theorem

Under some conditions and if $f(\mathbf{x})$ is convex, then the Newton's method converges in

$$\frac{f(\mathbf{x}^0) - p^*}{\gamma} + 6$$

steps (to reach accuracy of 5×10^{-20}). Here p^* is the optimal value of the problem, γ is a coefficient depending on the problem instance (see Boyd's book p490 for detail).

Newton's Method is Fast, But...

It requires computing the second-order derivative in each iteration

- It might be computationally expensive to do so, especially if the second-order derivative doesn't have a closed-form (which is the case for many useful applications)
- It also requires a lot of space to store the Hessian matrix.
  - If it is an n-dimensional problem, then we need n^2 space, comparing to n space required for gradient descent method
- It requires much more matrix computation in each iteration

One partial solution (quasi-Newton method)

- We don't update $\nabla^2 f(\mathbf{x})$ in each step. Instead, we only update it once in a while
- It can save a lot of computation, although one needs to be careful

Gradient Descent Method vs Newton's Method

First, the two methods have the same framework

Start from a point $\mathbf{x}^0$, set tolerance $\epsilon > 0$ and $k = 0$.

1. If $||\nabla f(\mathbf{x}^k)|| < \epsilon$, then output $\mathbf{x}^k$, otherwise compute the search direction $\mathbf{d}^k$ and continue;
2. Choose step size α_k by backtracking line search
3. Set $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$, $k = k + 1$, go back to step 1

In gradient descent method

$$\mathbf{d}^k = -\nabla f(\mathbf{x}^k)$$

In Newton's method

$$\mathbf{d}^k = -(\nabla^2 f(\mathbf{x}^k))^{-1} \nabla f(\mathbf{x}^k)$$

Tradeoff between Newton and Gradient Method

- Gradient method is fast in each iteration and needs less space, however, it takes much more iterations to converge
- Newton's method takes much less iterations to converge, however, each iteration is much more expensive in both computation and storage space

There are many numerical improvements that one can make but the main tradeoff remains

- Whether gradient method or Newton's method should be used depends largely on the problem at hand

Agenda

In the following, we will extend our discussions to constrained optimization problems.

- We will introduce one algorithm for such problems — projected gradient method

Constrained Optimization

We consider the following constrained optimization problem:

$$\begin{aligned} \text{minimize}_{\mathbf{x}} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & h_i(\mathbf{x}) = 0, \quad \forall i \\ & g_j(\mathbf{x}) \leq 0, \quad \forall j \end{aligned}$$

In unconstrained method, the main idea is:

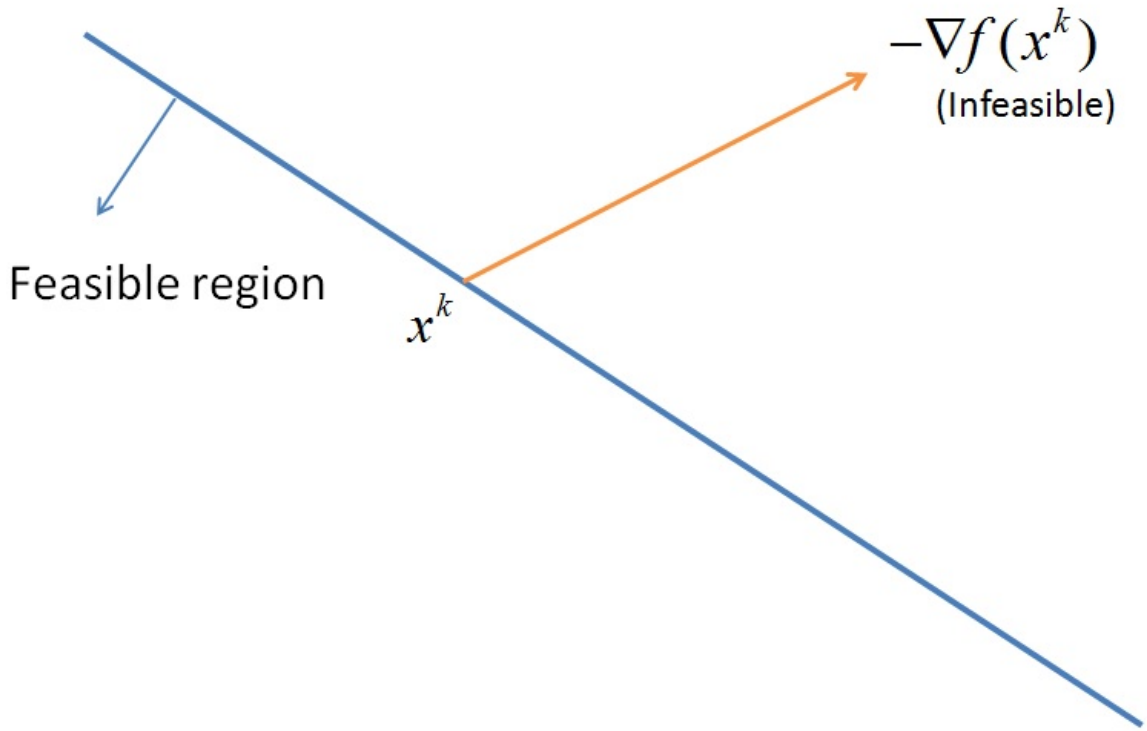
- At each $\mathbf{x}^k$, compute a descent direction, say $\mathbf{d}^k$
- Then find an appropriate step-size α_k and go to $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$
- Both gradient and Newton's methods are based on this idea

The problem when we have constraints

- $\mathbf{x}^{k+1}$ may become infeasible

In the following, we assume we have an initial feasible solution

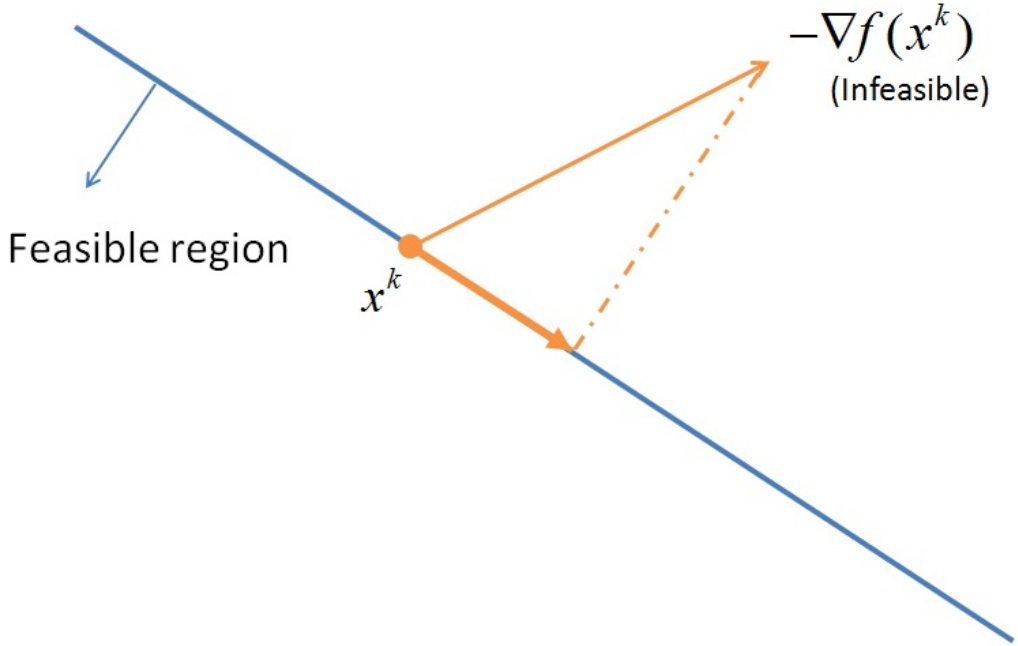
Illustration



- One of the solution to this problem is to use the projected gradient method

Main Idea of Projected Gradient Method

Project the descent direction onto the feasible set



We still use the general framework, however, the $\mathbf{d}^k$ chosen has to take into account feasibility.

Projected Gradient Method

We first consider a simpler case when the problem has linear equality constraints

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) \\ \text{subject to} \quad & A\mathbf{x} = \mathbf{b} \end{aligned}$$

Remember at $\mathbf{x}^k$, the feasible direction set is

$$S_F(\mathbf{x}^k) = \{\mathbf{d} | A\mathbf{d} = 0\}$$

Idea: We select a search direction that maintains feasibility while achieving maximum descent.

Projection

We solve the following *projection* problem:

$$\begin{aligned} \text{minimize}_{\mathbf{d}} \quad & \nabla f(\mathbf{x}^k)^T \mathbf{d} \\ \text{subject to} \quad & A\mathbf{d} = 0, ||\mathbf{d}||_2 \leq 1 \end{aligned} \tag{1}$$

If we don't have the constraint $A\mathbf{d} = 0$, then the optimal solution to the above problem will be

$$\mathbf{d}^* = \frac{-\nabla f(\mathbf{x}^k)}{||\nabla f(\mathbf{x}^k)||}$$

which is the same as in the gradient method.

With the constraint $A\mathbf{d} = 0$, one can view (1) as projecting the vector $-\nabla f(\mathbf{x}^k)$ to the set $\{\mathbf{d}|A\mathbf{d} = 0\}$

Projection

Theorem (Projection Theorem)

Given any vector $\mathbf{d}$*, the projection of* $\mathbf{d}$ *onto the region* $A\mathbf{d} = 0$ *is*

$$(I - A^T(AA^T)^{-1}A)\mathbf{d}$$

We call $P_A = I - A^T(AA^T)^{-1}A$ the projection matrix.

Theorem

Let $\mathbf{y} = P_A\mathbf{x}$*. Then* $\mathbf{y}$ *solves the following optimization problem:*

$$\begin{aligned} \textit{minimize}_{\mathbf{y}} \quad & ||\mathbf{y} - \mathbf{x}||_2^2 \\ s.t. \quad & A\mathbf{y} = 0 \end{aligned}$$

Proof Using KKT Conditions

Consider

$$\begin{aligned} \text{minimize}_{\mathbf{y}} \quad & ||\mathbf{y}-\mathbf{x}||_2^2 \\ \text{s.t.} \quad & A\mathbf{y}=0 \end{aligned}$$

Associate the constraints with multiplier λ, the KKT condition is:

$$2(\mathbf{y}-\mathbf{x})+A^T\lambda=0, \quad A\mathbf{y}=0$$

By multiplying A we get $\lambda=2(AA^T)^{-1}A\mathbf{x}$.

Plug it back in the KKT condition, we get

$$\mathbf{y}=\mathbf{x}-A^T(AA^T)^{-1}A\mathbf{x}=(I-A^T(AA^T)^{-1}A)\mathbf{x}=P_A\mathbf{x}$$

It is a convex optimization problem, thus $\mathbf{y}$ is the unique optimal solution. □

Properties of the Projection Matrix

There are several properties of P_A

- $AP_A = 0$ (therefore for any $\mathbf{x}$, $\mathbf{y} = P_A\mathbf{x}$ satisfies $A\mathbf{y} = 0$)
- $P_A^2 = P_A$
- All eigenvalues of P_A are either 0 or 1
- P_A is positive semi-definite, and $\mathbf{x}^T P_A \mathbf{x} = 0$ if and only if $P_A\mathbf{x} = 0$

Corollary

The optimal solution to the projection problem is

$$\bar{\mathbf{d}}^k = \frac{-P_A \nabla f(\mathbf{x}^k)}{||P_A \nabla f(\mathbf{x}^k)||_2}$$

In the projected gradient method, we will choose $\mathbf{d}^k = -P_A \nabla f(\mathbf{x}^k)$ as the search direction in each iteration.

Search Direction

We can verify that

$$\mathbf{d}^k = -P_A \nabla f(\mathbf{x}^k)$$

is a descent direction since

$$\nabla f(\mathbf{x}^k)^T \mathbf{d}^k = -\nabla f(\mathbf{x}^k)^T P_A \nabla f(\mathbf{x}^k) \leq 0$$

due to the positive semi-definiteness of P_A (it is < 0 as long as $P_A \nabla f(\mathbf{x}^k) \neq 0$)

- It is also a feasible direction since $A\mathbf{d}^k = 0$

Procedure

Procedure for projected gradient method with linear equality constraints:

Start with any point $\mathbf{x}^0$. Set $k = 0$ and stopping criterion $\epsilon > 0$. Define $P_A = I - A^T(AA^T)^{-1}A$

1. Check $||P_A \nabla f(\mathbf{x}^k)||$. If $||P_A \nabla f(\mathbf{x}^k)|| \leq \epsilon$, stop and output $\mathbf{x}^k$. Otherwise, continue to Step 2
2. Define $\mathbf{d}^k = -P_A \nabla f(\mathbf{x}^k)$ as the search direction
3. Use backtracking line search to find the step size α_k
4. Let $\mathbf{x}^{k+1} = \mathbf{x}^k + \alpha_k \mathbf{d}^k$, let $k = k + 1$. Go back to step 1

- Again, the only difference with the gradient or Newton's method is the search direction
- The reason for the difference is that we need to take care of feasibility in this case

Convergence Rate

Theorem (Convergence of Projected Gradient Method)

Under proper conditions, the projected gradient method converges to a local minimizer in a linear convergence rate.

Theorem (Projected Gradient for Convex Problems)

If the problem is convex, then under proper conditions, the projected gradient method converges to a global minimizer in a linear convergence rate.

Example

$$\begin{aligned} \text{minimize} \quad & e^{x_1+x_2} + x_1^2 + 3x_2^2 - x_1x_2 \\ \text{subject to} \quad & x_1 + 2x_2 = 1 \end{aligned}$$

The projection matrix is $I - A^T(AA^T)^{-1}A$ where $A = [1, 2]$. Thus, we have

$$P_A = \begin{bmatrix} 4/5 & -2/5 \\ -2/5 & 1/5 \end{bmatrix}$$

Use MATLAB to find the solution. Use the initial feasible solution $(1, 0)$

Example 2

$$\begin{aligned} \text{minimize}_{\mathbf{x}} \quad & ||A\mathbf{x} - \mathbf{b}||_2^2 \\ \text{subject to} \quad & W\mathbf{x} = \mathbf{z} \end{aligned}$$

Linear Inequality Constraints

If the constraints are:

$$\begin{aligned} \mathbf{a}_i^T \mathbf{x} &\leq b_i \quad i \in I_1 \\ \mathbf{a}_i^T \mathbf{x} &= b_i \quad i \in I_2 \end{aligned}$$

Then for any feasible solution $\mathbf{x}^k$, we first construct the feasible direction set:

$$\begin{aligned} \mathbf{a}_i^T \mathbf{d} &\leq 0 \quad \text{if } i \in I_1 \text{ and } a_i^T \mathbf{x}^k = b_i \\ \mathbf{a}_i^T \mathbf{d} &= 0 \qquad\qquad\quad i \in I_2 \end{aligned}$$

Linear Inequality Constraints

We consider a modified projection problem:

$$\begin{aligned} \text{minimize}_{\mathbf{d}} \quad & \nabla f(\mathbf{x}^k)^T \mathbf{d} \\ \text{subject to} \quad & \mathbf{a}_i^T \mathbf{d} \leq 0 \quad \text{if } i \in I_1 \text{ and } a_i^T \mathbf{x}^k = b_i \\ & \mathbf{a}_i^T \mathbf{d} = 0 \quad i \in I_2 \\ & ||\mathbf{d}||_2 \leq 1 \end{aligned}$$

The projection in this case is more complicated, but the idea is the same.

- However, the projection matrices will be different at each iteration, depending on which constraint is active or not
- The overall procedure is still the same, except the search directions at each step are different

Nonlinear Constraints

If the constraints are

$$g_j(\mathbf{x}) \leq 0 \quad \forall j$$

Then we can use a linear approximation for the feasible direction set:

$$\nabla g_j(\mathbf{x})^T \mathbf{d} \leq 0 \quad \text{if } g_j(\mathbf{x}) = 0$$

The feasible set is a linear function of $\mathbf{d}$ and therefore, we can solve a similar projection problem

Newton's Method with Linear Equality Constraints

The above discussion is all about extending the gradient descent methods to the constrained cases.

- We can also extend Newton's method to problems with constraints

The idea is similar

- Approximate the objective function by a quadratic function (using second-order Taylor expansion)
- Find the minimizer of the quadratic expansion, however, in this case, with the constraints

Newton's Method with Linear Equality Constraints

We consider

$$\begin{aligned} \text{minimize} \quad & f(\mathbf{x}) \\ \text{s.t.} \quad & A\mathbf{x} = \mathbf{b} \end{aligned}$$

At $\mathbf{x}^k$, compute the Newton's step by

$$\begin{aligned} \text{minimize}_{\mathbf{v}} \quad & f(\mathbf{x}^k) + \nabla f(\mathbf{x}^k)^T \mathbf{v} + \tfrac{1}{2}\mathbf{v}^T \nabla^2 f(\mathbf{x}^k)\mathbf{v} \\ \text{s.t.} \quad & A\mathbf{v} = 0 \end{aligned}$$

By KKT conditions, this equality constrained quadratic program can be solved explicitly:

$$\begin{bmatrix} \nabla^2 f(\mathbf{x}^k) & A^T \\ A & 0 \end{bmatrix} \begin{bmatrix} \mathbf{v} \\ \lambda \end{bmatrix} = \begin{bmatrix} -\nabla f(\mathbf{x}^k) \\ 0 \end{bmatrix}$$

where λ is the Lagrangian multiplier.

Newton's Method with Linear Equality Constraints

Therefore one can use $\mathbf{v}$ as the search direction, and use the same iterative framework

- Use backtracking search to find α and update

Similar idea can be applied to solve inequality constrained optimizations

- There might not be an explicit solution to the Newton's step
- However, it is just a quadratic program, which can be solved easily

To summarize, this approach projects the descent direction to the feasible direction set at each iteration, and otherwise proceeds the same as before.